JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class CompletableFuture<T>

java.lang.Object
    java.util.concurrent.CompletableFuture<T>

Type Parameters:

T - The result type returned by this future's `join` and `get` methods

All Implemented Interfaces:

`CompletionStage`<T>, `Future`<T>

```
public class CompletableFuture<T>
extends Object
implements Future<T>, CompletionStage<T>
```

A `Future` that may be explicitly completed (setting its value and status), and may be used as a `CompletionStage`, supporting dependent functions and actions that trigger upon its completion.

When two or more threads attempt to `complete`, `completeExceptionally`, or `cancel` a CompletableFuture, only one of them succeeds.

In addition to these and related methods for directly manipulating status and results, CompletableFuture implements interface `CompletionStage` with the following policies:

- Actions supplied for dependent completions of *non-async* methods may be performed by the thread that completes the current CompletableFuture, or by any other caller of a completion method.
- All *async* methods without an explicit Executor argument, as well as those involving delays are performed using the `ForkJoinPool.commonPool()`. The default async Executor may be overridden for non-static methods in subclasses by defining method `defaultExecutor()`. To simplify monitoring, debugging, and tracking, all generated asynchronous tasks are instances of the marker interface `CompletableFuture.AsynchronousCompletionTask`. Operations with time-delays can use adapter methods defined in this class, for example: `supplyAsync(supplier, delayedExecutor(timeout, timeUnit))`.
- All CompletionStage methods are implemented independently of other public methods, so the behavior of one method is not impacted by overrides of others in subclasses.
- All CompletionStage methods return CompletableFutures. To restrict usages to only those methods defined in interface CompletionStage, use method `minimalCompletionStage()`. Or to ensure only that clients do not themselves modify a future, use method `copy()`.

CompletableFuture also implements `Future` with the following policies:

- Since (unlike `FutureTask`) this class has no direct control over the computation that causes it to be completed, cancellation is treated as just another form of exceptional completion. Method `cancel` has the same effect as `completeExceptionally(new CancellationException())`. Method `isCompletedExceptionally()` can be used to determine if a CompletableFuture completed in any exceptional fashion.
- In case of exceptional completion with a CompletionException, methods `get()` and `get(long, TimeUnit)` throw an `ExecutionException` with the same cause as held in the corresponding CompletionException. To simplify usage in most contexts, this class also defines methods `join()` and `getNow(T)` that instead throw the CompletionException directly in these cases.

Arguments used to pass a completion result (that is, for parameters of type T) for methods accepting them may be null, but passing a null value for any other parameter will result in a `NullPointerException` being thrown.

Subclasses of this class should normally override the "virtual constructor" method `newIncompleteFuture()`, which establishes the concrete type returned by CompletionStage methods. For example, here is a class that substitutes a different default Executor and disables the `obtrude` methods:

```
class MyCompletableFuture<T> extends CompletableFuture<T> {
  static final Executor myExecutor = ...;
  public MyCompletableFuture() { }
  public <U> CompletableFuture<U> newIncompleteFuture() {
    return new MyCompletableFuture<U>(); }
  public Executor defaultExecutor() {
    return myExecutor; }
  public void obtrudeValue(T value) {
    throw new UnsupportedOperationException(); }
  public void obtrudeException(Throwable ex) {
    throw new UnsupportedOperationException(); }
}
```

Since:

1.8

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description

static interface	CompletableFuture.Asynch	A marker interface identifying asynchronous tasks produced by async methods.

Nested classes/interfaces declared in interface Future

Future.State

Constructor Summary

Constructors

Constructor	Description
CompletableFuture()	Creates a new incomplete CompletableFuture.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
CompletableFuture<Void>	acceptEither(CompletionStage<? extends T> other, Consumer<? super T> action)	Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed with the corresponding result as argument to the supplied action.
CompletableFuture<Void>	acceptEitherAsync(CompletionStage<? extends T> other, Consumer<? super T> action)	Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the corresponding result as

		argument to the supplied action.
CompletableFuture<Void>	**acceptEitherAsync**(CompletionStage<? extends T> other, Consumer<? super T> action, Executor executor)	Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied action.
static CompletableFuture<Void>	**allOf**(CompletableFuture<?>... cfs)	Returns a new CompletableFuture that is completed when all of the given CompletableFutures complete.
static CompletableFuture<Object>	**anyOf**(CompletableFuture<?>... cfs)	Returns a new CompletableFuture that is completed when any of the given CompletableFutures complete, with the same result.
<U> CompletableFutur<U>	**applyToEither**(CompletionStage<? extends T> other, Function<? super T, U> fn)	Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed with the

		corresponding result as argument to the supplied function.
<U> CompletableFutur <U>	**applyToEitherAsync** (CompletionStage<? extends T> other, Function<? super T, U> fn)	Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the corresponding result as argument to the supplied function.
<U> CompletableFutur <U>	**applyToEitherAsync** (CompletionStage<? extends T> other, Function<? super T, U> fn, Executor executor)	Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied function.
boolean	**cancel** (boolean mayInterruptIfR	If not already completed, completes this CompletableFuture with a CancellationException.
boolean	**complete**(T value)	If not already completed, sets the value returned by get() and related methods to the given value.

CompletableFuture<T>	**completeAsync**(Supplier<? extends T> supplier)	Completes this CompletableFuture with the result of the given Supplier function invoked from an asynchronous task using the default executor.
CompletableFuture<T>	**completeAsync**(Supplier<? extends T> supplier, Executor executor)	Completes this CompletableFuture with the result of the given Supplier function invoked from an asynchronous task using the given executor.
static <U> CompletableFutur<U>	**completedFuture**(U value)	Returns a new CompletableFuture that is already completed with the given value.
static <U> CompletionStage<U>	**completedStage**(U value)	Returns a new CompletionStage that is already completed with the given value and supports only those methods in interface CompletionStage.
boolean	**completeExceptionally**(Throwable ex)	If not already completed, causes invocations of get() and related methods to throw the given exception.
CompletableFuture<T>	**completeOnTimeout**(T value, long timeout, TimeUnit unit)	Completes this CompletableFuture with the given value if not otherwise completed before the given timeout elapsed.
CompletableFuture<T>	**copy**()	Returns a new CompletableFuture that is completed normally with the same value as

		this CompletableFuture when it completes normally.
Executor	**defaultExecutor**()	Returns the default Executor used for async methods that do not specify an Executor.
static Executor	**delayedExecutor**(long delay, TimeUnit unit)	Returns a new Executor that submits a task to the default executor after the given delay (or no delay if non-positive).
static Executor	**delayedExecutor**(long delay, TimeUnit unit, Executor executor)	Returns a new Executor that submits a task to the given base executor after the given delay (or no delay if non-positive).
CompletableFuture<T>	**exceptionally**(Function<Throwable, ? extends T> fn)	Returns a new CompletionStage that, when this stage completes exceptionally, is executed with this stage's exception as the argument to the supplied function.
CompletableFuture<T>	**exceptionallyAsync**(Function<Throwable, ? extends T> fn)	Returns a new CompletionStage that, when this stage completes exceptionally, is executed with this stage's exception as the argument to the supplied function, using this stage's default asynchronous execution facility.

CompletableFuture<T>	**exceptionallyAsync**(Function<Throwable, ? extends T> fn, Executor executor)	Returns a new CompletionStage that, when this stage completes exceptionally, is executed with this stage's exception as the argument to the supplied function, using the supplied Executor.
CompletableFuture<T>	**exceptionallyCompose**(Function<Throwable, ? extends CompletionStage<T>> fn)	Returns a new CompletionStage that, when this stage completes exceptionally, is composed using the results of the supplied function applied to this stage's exception.
CompletableFuture<T>	**exceptionallyComposeAsyn**(Function<Throwable, ? extends CompletionStage<T>> fn)	Returns a new CompletionStage that, when this stage completes exceptionally, is composed using the results of the supplied function applied to this stage's exception, using this stage's default asynchronous execution facility.
CompletableFuture<T>	**exceptionallyComposeAsyn**(Function<Throwable, ? extends CompletionStage<T>> fn, Executor executor)	Returns a new CompletionStage that, when this stage completes exceptionally, is composed using the results of the supplied function applied to this stage's exception, using the supplied Executor.
Throwable	**exceptionNow**()	Returns the exception thrown by the task, without waiting.

static <U> CompletableFutur <U>	**failedFuture** (Throwable ex)	Returns a new CompletableFuture that is already completed exceptionally with the given exception.
static <U> CompletionStage <U>	**failedStage** (Throwable ex)	Returns a new CompletionStage that is already completed exceptionally with the given exception and supports only those methods in interface CompletionStage.
T	**get**()	Waits if necessary for this future to complete, and then returns its result.
T	**get**(long timeout, TimeUnit unit)	Waits if necessary for at most the given time for this future to complete, and then returns its result, if available.
T	**getNow**(T valueIfAbsent)	Returns the result value (or throws any encountered exception) if completed, else returns the given valueIfAbsent.
int	**getNumberOfDependents**()	Returns the estimated number of CompletableFutures whose completions are awaiting completion of this CompletableFuture.
<U> CompletableFutur <U>	**handle**(BiFunction<? super T, Throwable, ? extends U> fn)	Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is

		executed with `this` stage's result and exception as arguments to the supplied function.
`<U> CompletableFutur<U>`	**`handleAsync`**`(BiFunction<? super T, Throwable, ? extends U> fn)`	Returns a new CompletionStage that, when `this` stage completes either normally or exceptionally, is executed using `this` stage's default asynchronous execution facility, with `this` stage's result and exception as arguments to the supplied function.
`<U> CompletableFutur<U>`	**`handleAsync`**`(BiFunction<? super T, Throwable, ? extends U> fn, Executor executor)`	Returns a new CompletionStage that, when `this` stage completes either normally or exceptionally, is executed using the supplied executor, with `this` stage's result and exception as arguments to the supplied function.
`boolean`	**`isCancelled`**`()`	Returns `true` if this CompletableFuture was cancelled before it completed normally.
`boolean`	**`isCompletedExceptionally`**	Returns `true` if this CompletableFuture

		completed exceptionally, in any way.
boolean	**isDone**()	Returns true if completed in any fashion: normally, exceptionally, or via cancellation.
T	**join**()	Returns the result value when complete, or throws an (unchecked) exception if completed exceptionally.
CompletionStage<T>	**minimalCompletionStage**()	Returns a new CompletionStage that is completed normally with the same value as this CompletableFuture when it completes normally, and cannot be independently completed or otherwise used in ways not defined by the methods of interface CompletionStage.
<U> CompletableFutur <U>	**newIncompleteFuture**()	Returns a new incomplete CompletableFuture of the type to be returned by a CompletionStage method.
void	**obtrudeException** (Throwable ex)	Forcibly causes subsequent invocations of method get() and related methods to throw the given exception, whether or not already completed.
void	**obtrudeValue**(T value)	

		Forcibly sets or resets the value subsequently returned by method `get()` and related methods, whether or not already completed.
`CompletableFuture<T>`	**`orTimeout`**`(long timeout, TimeUnit unit)`	Exceptionally completes this CompletableFuture with a `TimeoutException` if not otherwise completed before the given timeout elapsed.
`T`	**`resultNow`**`()`	Returns the computed result, without waiting.
`CompletableFuture<Void>`	**`runAfterBoth`**`(CompletionStage<?> other, Runnable action)`	Returns a new CompletionStage that, when `this` and the other given stage both complete normally, executes the given action.
`CompletableFuture<Void>`	**`runAfterBothAsync`**`(CompletionStage<?> other, Runnable action)`	Returns a new CompletionStage that, when `this` and the other given stage both complete normally, executes the given action using `this` stage's default asynchronous execution facility.
`CompletableFuture<Void>`	**`runAfterBothAsync`**`(CompletionStage<?> other, Runnable action, Executor executor)`	Returns a new CompletionStage that, when `this` and the other given stage both complete normally, executes the given action using the supplied executor.
	`runAfterEither``(CompletionStage<?`	Returns a new CompletionStage that,

CompletableFuture<Void>	> other, Runnable action)	when either this or the other given stage complete normally, executes the given action.
CompletableFuture<Void>	**runAfterEitherAsync**(CompletionStage<? > other, Runnable action)	Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action using this stage's default asynchronous execution facility.
CompletableFuture<Void>	**runAfterEitherAsync**(CompletionStage<? > other, Runnable action, Executor executor)	Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action using the supplied executor.
static CompletableFuture<Void>	**runAsync**(Runnable runnable)	Returns a new CompletableFuture that is asynchronously completed by a task running in the ForkJoinPool.commonPool() after it runs the given action.
static CompletableFuture<Void>	**runAsync**(Runnable runnable, Executor executor)	Returns a new CompletableFuture that is asynchronously completed by a task running in the given executor after it runs the given action.
Future.State	**state**()	

		Returns the computation state.
static <U> CompletableFutur <U>	**supplyAsync** (Supplier<U> supplier)	Returns a new CompletableFuture that is asynchronously completed by a task running in the ForkJoinPool.commonPool() with the value obtained by calling the given Supplier.
static <U> CompletableFutur <U>	**supplyAsync** (Supplier<U> supplier, Executor executor)	Returns a new CompletableFuture that is asynchronously completed by a task running in the given executor with the value obtained by calling the given Supplier.
CompletableFuture <Void>	**thenAccept**(Consumer<? super T> action)	Returns a new CompletionStage that, when this stage completes normally, is executed with this stage's result as the argument to the supplied action.
CompletableFuture <Void>	**thenAcceptAsync** (Consumer<? super T> action)	Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied action.
CompletableFuture <Void>	**thenAcceptAsync** (Consumer<? super	Returns a new CompletionStage that, when this stage

	T> action, Executor executor)	completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied action.
<U> CompletableFutur <Void>	**thenAcceptBoth** (CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action)	Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed with the two results as arguments to the supplied action.
<U> CompletableFutur <Void>	**thenAcceptBothAsync** (CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action)	Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed using this stage's default asynchronous execution facility, with the two results as arguments to the supplied action.
<U> CompletableFutur <Void>	**thenAcceptBothAsync** (CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action, Executor executor)	Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed using the supplied executor, with the two results as arguments to the supplied action.
<U> CompletableFutur <U>	**thenApply**(Function<? super T, ? extends U> fn)	Returns a new CompletionStage that, when this stage

		completes normally, is executed with this stage's result as the argument to the supplied function.
<U> CompletableFutur <U>	**thenApplyAsync** (Function<? super T, ? extends U> fn)	Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied function.
<U> CompletableFutur <U>	**thenApplyAsync** (Function<? super T, ? extends U> fn, Executor executor)	Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied function.
<U, V> CompletableFuture <V>	**thenCombine** (CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn)	Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed with the two results as arguments to the supplied function.
<U, V> CompletableFuture <V>	**thenCombineAsync** (CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn)	Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed using this

		stage's default asynchronous execution facility, with the two results as arguments to the supplied function.
<U, V> CompletableFuture<V>	**thenCombineAsync**(CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn, Executor executor)	Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed using the supplied executor, with the two results as arguments to the supplied function.
<U> CompletableFutur<U>	**thenCompose**(Function<? super T, ? extends CompletionStage<U>> fn)	Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function.
<U> CompletableFutur<U>	**thenComposeAsync**(Function<? super T, ? extends CompletionStage<U>> fn)	Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function, executed using this stage's default asynchronous execution facility.
<U> CompletableFutur<U>	**thenComposeAsync**(Function<? super T, ? extends	Returns a new CompletionStage that is completed with the same

	CompletionStage<U>> fn, Executor executor)	value as the CompletionStage returned by the given function, executed using the supplied Executor.
CompletableFuture<Void>	**thenRun**(Runnable action)	Returns a new CompletionStage that, when this stage completes normally, executes the given action.
CompletableFuture<Void>	**thenRunAsync**(Runnable action)	Returns a new CompletionStage that, when this stage completes normally, executes the given action using this stage's default asynchronous execution facility.
CompletableFuture<Void>	**thenRunAsync**(Runnable action, Executor executor)	Returns a new CompletionStage that, when this stage completes normally, executes the given action using the supplied Executor.
CompletableFuture<T>	**toCompletableFuture**()	Returns this CompletableFuture.
String	**toString**()	Returns a string identifying this CompletableFuture, as well as its completion state.
CompletableFuture<T>	**whenComplete**(BiConsumer<? super	Returns a new CompletionStage with the same result or exception as this stage,

	T, ? super Throwable> action)	that executes the given action when this stage completes.
CompletableFuture<T>	**whenCompleteAsync**(BiConsumer<? super T, ? super Throwable> action)	Returns a new CompletionStage with the same result or exception as this stage, that executes the given action using this stage's default asynchronous execution facility when this stage completes.
CompletableFuture<T>	**whenCompleteAsync**(BiConsumer<? super T, ? super Throwable> action, Executor executor)	Returns a new CompletionStage with the same result or exception as this stage, that executes the given action using the supplied Executor when this stage completes.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Details

CompletableFuture

```
public CompletableFuture()
```

Creates a new incomplete CompletableFuture.

Method Details

supplyAsync

```
public static <U> CompletableFuture<U> supplyAsync
(Supplier<U> supplier)
```

Returns a new CompletableFuture that is asynchronously completed by a task running in the `ForkJoinPool.commonPool()` with the value obtained by calling the given Supplier.

Type Parameters:

`U` - the function's return type

Parameters:

`supplier` - a function returning the value to be used to complete the returned CompletableFuture

Returns:

the new CompletableFuture

supplyAsync

```
public static <U> CompletableFuture<U> supplyAsync
(Supplier<U> supplier,
 Executor executor)
```

Returns a new CompletableFuture that is asynchronously completed by a task running in the given executor with the value obtained by calling the given Supplier.

Type Parameters:

`U` - the function's return type

Parameters:

`supplier` - a function returning the value to be used to complete the returned CompletableFuture

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletableFuture

runAsync

```
public static CompletableFuture<Void> runAsync(Runnable runnable)
```

Returns a new CompletableFuture that is asynchronously completed by a task running in the `ForkJoinPool.commonPool()` after it runs the given action.

Parameters:

`runnable` - the action to run before completing the returned CompletableFuture

Returns:

the new CompletableFuture

runAsync

```
public static CompletableFuture<Void> runAsync(Runnable runnable,
                                               Executor executor)
```

Returns a new CompletableFuture that is asynchronously completed by a task running in the given executor after it runs the given action.

Parameters:

`runnable` - the action to run before completing the returned CompletableFuture

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletableFuture

completedFuture

```
public static <U> CompletableFuture<U> completedFuture(U value)
```

Returns a new CompletableFuture that is already completed with the given value.

Type Parameters:

`U` - the type of the value

Parameters:

`value` - the value

Returns:

the completed CompletableFuture

isDone

```
public boolean isDone()
```

Returns `true` if completed in any fashion: normally, exceptionally, or via cancellation.

Specified by:

`isDone` in interface `Future<T>`

Returns:

`true` if completed

get

```
public T get()
      throws InterruptedException,
             ExecutionException
```

Waits if necessary for this future to complete, and then returns its result.

Specified by:

`get` in interface `Future<T>`

Returns:

the result value

Throws:

`CancellationException` - if this future was cancelled

`ExecutionException` - if this future completed exceptionally

`InterruptedException` - if the current thread was interrupted while waiting

get

```
public T get(long timeout,
             TimeUnit unit)
      throws InterruptedException,
             ExecutionException,
             TimeoutException
```

Waits if necessary for at most the given time for this future to complete, and then returns its result, if available.

Specified by:

`get` in interface `Future<T>`

Parameters:

`timeout` - the maximum time to wait

`unit` - the time unit of the timeout argument

Returns:

the result value

Throws:

`CancellationException` - if this future was cancelled

`ExecutionException` - if this future completed exceptionally

`InterruptedException` - if the current thread was interrupted while waiting

`TimeoutException` - if the wait timed out

join

`public T join()`

Returns the result value when complete, or throws an (unchecked) exception if completed exceptionally. To better conform with the use of common functional forms, if a computation involved in the completion of this CompletableFuture threw an exception, this method throws an (unchecked) `CompletionException` with the underlying exception as its cause.

Returns:

the result value

Throws:

`CancellationException` - if the computation was cancelled

`CompletionException` - if this future completed exceptionally or a completion computation threw an exception

getNow

`public T getNow(T valueIfAbsent)`

Returns the result value (or throws any encountered exception) if completed, else returns the given valueIfAbsent.

Parameters:

`valueIfAbsent` - the value to return if not completed

Returns:

the result value, if completed, else the given valueIfAbsent

Throws:

`CancellationException` - if the computation was cancelled

`CompletionException` - if this future completed exceptionally or a completion computation threw an exception

resultNow

`public T resultNow()`

Description copied from interface: `Future`

Returns the computed result, without waiting.

This method is for cases where the caller knows that the task has already completed successfully, for example when filtering a stream of Future objects for the successful tasks and using a mapping operation to obtain a stream of results.

```
    results = futures.stream()
                .filter(f -> f.state() == Future.State.SUCCESS)
                .map(Future::resultNow)
                .toList();
```

Specified by:

`resultNow` in interface `Future<T>`

Returns:

the computed result

Since:

19

exceptionNow

```
public Throwable exceptionNow()
```

Description copied from interface: `Future`

Returns the exception thrown by the task, without waiting.

This method is for cases where the caller knows that the task has already completed with an exception.

Specified by:

`exceptionNow` in interface `Future<T>`

Returns:

the exception thrown by the task

Since:

19

complete

```
public boolean complete(T value)
```

If not already completed, sets the value returned by `get()` and related methods to the given value.

Parameters:

`value` - the result value

Returns:

`true` if this invocation caused this CompletableFuture to transition to a completed state, else `false`

completeExceptionally

```
public boolean completeExceptionally(Throwable ex)
```

If not already completed, causes invocations of `get()` and related methods to throw the given exception.

Parameters:

`ex` - the exception

Returns:

`true` if this invocation caused this CompletableFuture to transition to a completed state, else `false`

thenApply

```
public <U> CompletableFuture<U> thenApply
(Function<? super T, ? extends U> fn)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` stage completes normally, is executed with `this` stage's result as the argument to the supplied function.

This method is analogous to `Optional.map` and `Stream.map`.

See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`thenApply` in interface `CompletionStage<T>`

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenApplyAsync

```
public <U> CompletableFuture<U> thenApplyAsync
(Function<? super T, ? extends U> fn)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` stage completes normally, is executed using `this` stage's default asynchronous execution facility, with `this`

stage's result as the argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`thenApplyAsync` in interface `CompletionStage<T>`

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenApplyAsync

```
public <U> CompletableFuture<U> thenApplyAsync
(Function<? super T, ? extends U> fn,
 Executor executor)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` stage completes normally, is executed using the supplied Executor, with `this` stage's result as the argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`thenApplyAsync` in interface `CompletionStage<T>`

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenAccept

```
public CompletableFuture<Void> thenAccept(Consumer<? super T> action)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` stage completes normally, is executed with `this` stage's result as the argument to the supplied action. See

the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenAccept in interface CompletionStage<T>

Parameters:

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptAsync

```
public CompletableFuture<Void> thenAcceptAsync
(Consumer<? super T> action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenAcceptAsync in interface CompletionStage<T>

Parameters:

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptAsync

```
public CompletableFuture<Void> thenAcceptAsync
(Consumer<? super T> action,
 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenAcceptAsync in interface CompletionStage<T>

Parameters:

`action` - the action to perform before completing the returned CompletionStage

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenRun

```
public CompletableFuture<Void> thenRun(Runnable action)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` stage completes normally, executes the given action. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`thenRun` in interface `CompletionStage<T>`

Parameters:

`action` - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenRunAsync

```
public CompletableFuture<Void> thenRunAsync(Runnable action)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` stage completes normally, executes the given action using `this` stage's default asynchronous execution facility. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`thenRunAsync` in interface `CompletionStage<T>`

Parameters:

`action` - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenRunAsync

```
public CompletableFuture<Void> thenRunAsync(Runnable action,
                                            Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, executes the given action using the supplied Executor. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenRunAsync in interface CompletionStage<T>

Parameters:

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenCombine

```
public <U,V> CompletableFuture<V> thenCombine
(CompletionStage<? extends U> other,
 BiFunction<? super T, ? super U, ? extends V> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed with the two results as arguments to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenCombine in interface CompletionStage<T>

Type Parameters:

U - the type of the other CompletionStage's result

V - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenCombineAsync

```
public <U,V> CompletableFuture<V> thenCombineAsync
(CompletionStage<? extends U> other,
 BiFunction<? super T, ? super U, ? extends V> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed using `this` stage's default asynchronous execution facility, with the two results as arguments to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`thenCombineAsync` in interface `CompletionStage<T>`

Type Parameters:

`U` - the type of the other CompletionStage's result

`V` - the function's return type

Parameters:

`other` - the other CompletionStage

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenCombineAsync

```
public <U,V> CompletableFuture<V> thenCombineAsync
(CompletionStage<? extends U> other,
 BiFunction<? super T, ? super U, ? extends V> fn,
 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed using the supplied executor, with the two results as arguments to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`thenCombineAsync` in interface `CompletionStage<T>`

Type Parameters:

`U` - the type of the other CompletionStage's result

`V` - the function's return type

Parameters:

`other` - the other CompletionStage

`fn` - the function to use to compute the value of the returned CompletionStage

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenAcceptBoth

```
public <U> CompletableFuture<Void> thenAcceptBoth
(CompletionStage<? extends U> other,
 BiConsumer<? super T, ? super U> action)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed with the two results as arguments to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`thenAcceptBoth` in interface `CompletionStage<T>`

Type Parameters:

`U` - the type of the other CompletionStage's result

Parameters:

`other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptBothAsync

```
public <U> CompletableFuture<Void> thenAcceptBothAsync
(CompletionStage<? extends U> other,
 BiConsumer<? super T, ? super U> action)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed using `this` stage's default asynchronous execution facility, with the two results as arguments to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

thenAcceptBothAsync in interface CompletionStage<T>

Type Parameters:

U - the type of the other CompletionStage's result

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptBothAsync

```
public <U> CompletableFuture<Void> thenAcceptBothAsync
(CompletionStage<? extends U> other,
 BiConsumer<? super T, ? super U> action,
 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed using the supplied executor, with the two results as arguments to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenAcceptBothAsync in interface CompletionStage<T>

Type Parameters:

U - the type of the other CompletionStage's result

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

runAfterBoth

```
public CompletableFuture<Void> runAfterBoth(CompletionStage<?> other,
                                            Runnable action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when `this` and the other given stage both complete normally, executes the given action. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`runAfterBoth` in interface `CompletionStage<T>`

Parameters:

`other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterBothAsync

```
public CompletableFuture<Void> runAfterBothAsync
(CompletionStage<?> other,
 Runnable action)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` and the other given stage both complete normally, executes the given action using `this` stage's default asynchronous execution facility. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`runAfterBothAsync` in interface `CompletionStage<T>`

Parameters:

`other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterBothAsync

```
public CompletableFuture<Void> runAfterBothAsync
(CompletionStage<?> other,
 Runnable action,
 Executor executor)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` and the other given stage both complete normally, executes the given action using the supplied

executor. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

runAfterBothAsync in interface CompletionStage<T>

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

applyToEither

```
public <U> CompletableFuture<U> applyToEither
(CompletionStage<? extends T> other,
 Function<? super T, U> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed with the corresponding result as argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

applyToEither in interface CompletionStage<T>

Type Parameters:

U - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

applyToEitherAsync

```
public <U> CompletableFuture<U> applyToEitherAsync
(CompletionStage<? extends T> other,
 Function<? super T, U> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed using `this` stage's default asynchronous execution facility, with the corresponding result as argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`applyToEitherAsync` in interface `CompletionStage<T>`

Type Parameters:

`U` - the function's return type

Parameters:

`other` - the other CompletionStage

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

applyToEitherAsync

```
public <U> CompletableFuture<U> applyToEitherAsync
(CompletionStage<? extends T> other,
 Function<? super T, U> fn,
 Executor executor)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`applyToEitherAsync` in interface `CompletionStage<T>`

Type Parameters:

`U` - the function's return type

Parameters:

`other` - the other CompletionStage

`fn` - the function to use to compute the value of the returned CompletionStage

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

acceptEither

```
public CompletableFuture<Void> acceptEither
(CompletionStage<? extends T> other,
 Consumer<? super T> action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed with the corresponding result as argument to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`acceptEither` in interface `CompletionStage<T>`

Parameters:

`other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

acceptEitherAsync

```
public CompletableFuture<Void> acceptEitherAsync
(CompletionStage<? extends T> other,
 Consumer<? super T> action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed using `this` stage's default asynchronous execution facility, with the corresponding result as argument to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`acceptEitherAsync` in interface `CompletionStage<T>`

Parameters:

`other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

acceptEitherAsync

```
public CompletableFuture<Void> acceptEitherAsync
(CompletionStage<? extends T> other,
 Consumer<? super T> action,
 Executor executor)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`acceptEitherAsync` in interface `CompletionStage<T>`

Parameters:

`other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

runAfterEither

```
public CompletableFuture<Void> runAfterEither
(CompletionStage<?> other,
 Runnable action)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when either `this` or the other given stage complete normally, executes the given action. See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`runAfterEither` in interface `CompletionStage<T>`

Parameters:

`other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterEitherAsync

```
public CompletableFuture<Void> runAfterEitherAsync
(CompletionStage<?> other,
 Runnable action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either `this` or the other given stage complete normally, executes the given action using `this` stage's default asynchronous execution facility. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

runAfterEitherAsync in interface CompletionStage<T>

Parameters:

`other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterEitherAsync

```
public CompletableFuture<Void> runAfterEitherAsync
(CompletionStage<?> other,
 Runnable action,
 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either `this` or the other given stage complete normally, executes the given action using the supplied executor. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

runAfterEitherAsync in interface CompletionStage<T>

Parameters:

`other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenCompose

```
public <U> CompletableFuture<U> thenCompose
(Function<? super T, ? extends CompletionStage<U>> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function.

When `this` stage completes normally, the given function is invoked with `this` stage's result as the argument, returning another CompletionStage. When that stage completes normally, the CompletionStage returned is completed with the same value.

To ensure progress, the supplied function must arrange eventual completion of its result.

This method is analogous to `Optional.flatMap` and `Stream.flatMap`.

See the `CompletionStage` documentation for rules covering exceptional completion.

Specified by:

`thenCompose` in interface `CompletionStage<T>`

Type Parameters:

`U` - the type of the returned CompletionStage's result

Parameters:

`fn` - the function to use to compute another CompletionStage

Returns:

the new CompletionStage

thenComposeAsync

```
public <U> CompletableFuture<U> thenComposeAsync
(Function<? super T, ? extends CompletionStage<U>> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function, executed using `this` stage's default asynchronous execution facility.

When `this` stage completes normally, the given function is invoked with `this` stage's result as the argument, returning another CompletionStage. When that stage completes normally, the CompletionStage returned is completed with the same value.

To ensure progress, the supplied function must arrange eventual completion of its result.

See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenComposeAsync in interface CompletionStage<T>

Type Parameters:

U - the type of the returned CompletionStage's result

Parameters:

fn - the function to use to compute another CompletionStage

Returns:

the new CompletionStage

thenComposeAsync

```
public <U> CompletableFuture<U> thenComposeAsync
(Function<? super T, ? extends CompletionStage<U>> fn,
 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function, executed using the supplied Executor.

When this stage completes normally, the given function is invoked with this stage's result as the argument, returning another CompletionStage. When that stage completes normally, the CompletionStage returned by this method is completed with the same value.

To ensure progress, the supplied function must arrange eventual completion of its result.

See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenComposeAsync in interface CompletionStage<T>

Type Parameters:

U - the type of the returned CompletionStage's result

Parameters:

fn - the function to use to compute another CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

whenComplete

```
public CompletableFuture<T> whenComplete
(BiConsumer<? super T, ? super Throwable> action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage with the same result or exception as `this` stage, that executes the given action when `this` stage completes.

When `this` stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of `this` stage as arguments. The returned stage is completed when the action returns.

Unlike method `handle`, method `whenComplete` is not designed to translate completion outcomes, so the supplied action should not throw an exception. However, if it does, the following rules apply: if `this` stage completed normally but the supplied action throws an exception, then the returned stage completes exceptionally with the supplied action's exception. Or, if `this` stage completed exceptionally and the supplied action throws an exception, then the returned stage completes exceptionally with `this` stage's exception.

Specified by:

`whenComplete` in interface `CompletionStage<T>`

Parameters:

`action` - the action to perform

Returns:

the new CompletionStage

whenCompleteAsync

```
public CompletableFuture<T> whenCompleteAsync
(BiConsumer<? super T, ? super Throwable> action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage with the same result or exception as `this` stage, that executes the given action using `this` stage's default asynchronous execution facility when `this` stage completes.

When `this` stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of `this` stage as arguments. The returned stage is completed when the action returns.

Unlike method handleAsync, method whenCompleteAsync is not designed to translate completion outcomes, so the supplied action should not throw an exception. However, if it does, the following rules apply: If this stage completed normally but the supplied action throws an exception, then the returned stage completes exceptionally with the supplied action's exception. Or, if this stage completed exceptionally and the supplied action throws an exception, then the returned stage completes exceptionally with this stage's exception.

Specified by:

whenCompleteAsync in interface CompletionStage<T>

Parameters:

action - the action to perform

Returns:

the new CompletionStage

whenCompleteAsync

```
public CompletableFuture<T> whenCompleteAsync
(BiConsumer<? super T, ? super Throwable> action,
 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage with the same result or exception as this stage, that executes the given action using the supplied Executor when this stage completes.

When this stage is complete, the given action is invoked with the result (or null if none) and the exception (or null if none) of this stage as arguments. The returned stage is completed when the action returns.

Unlike method handleAsync, method whenCompleteAsync is not designed to translate completion outcomes, so the supplied action should not throw an exception. However, if it does, the following rules apply: If this stage completed normally but the supplied action throws an exception, then the returned stage completes exceptionally with the supplied action's exception. Or, if this stage completed exceptionally and the supplied action throws an exception, then the returned stage completes exceptionally with this stage's exception.

Specified by:

whenCompleteAsync in interface CompletionStage<T>

Parameters:

action - the action to perform

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

handle

```
public <U> CompletableFuture<U> handle
(BiFunction<? super T, Throwable, ? extends U> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed with this stage's result and exception as arguments to the supplied function.

When this stage is complete, the given function is invoked with the result (or null if none) and the exception (or null if none) of this stage as arguments, and the function's result is used to complete the returned stage.

Specified by:

handle in interface CompletionStage<T>

Type Parameters:

U - the function's return type

Parameters:

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

handleAsync

```
public <U> CompletableFuture<U> handleAsync
(BiFunction<? super T, Throwable, ? extends U> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed using this stage's default asynchronous execution facility, with this stage's result and exception as arguments to the supplied function.

When this stage is complete, the given function is invoked with the result (or null if none) and the exception (or null if none) of this stage as arguments, and the function's result is used to complete the returned stage.

Specified by:

handleAsync in interface CompletionStage<T>

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

handleAsync

```
public <U> CompletableFuture<U> handleAsync
(BiFunction<? super T, Throwable, ? extends U> fn,
 Executor executor)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` stage completes either normally or exceptionally, is executed using the supplied executor, with `this` stage's result and exception as arguments to the supplied function.

When `this` stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of `this` stage as arguments, and the function's result is used to complete the returned stage.

Specified by:

`handleAsync` in interface `CompletionStage<T>`

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

toCompletableFuture

```
public CompletableFuture<T> toCompletableFuture()
```

Returns this CompletableFuture.

Specified by:

`toCompletableFuture` in interface `CompletionStage<T>`

Returns:

this CompletableFuture

exceptionally

```
public CompletableFuture<T> exceptionally
(Function<Throwable, ? extends T> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when `this` stage completes exceptionally, is executed with `this` stage's exception as the argument to the supplied function. Otherwise, if `this` stage completes normally, then the returned stage also completes normally with the same value.

Specified by:

`exceptionally` in interface `CompletionStage<T>`

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage if `this` CompletionStage completed exceptionally

Returns:

the new CompletionStage

exceptionallyAsync

```
public CompletableFuture<T> exceptionallyAsync
(Function<Throwable, ? extends T> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when `this` stage completes exceptionally, is executed with `this` stage's exception as the argument to the supplied function, using `this` stage's default asynchronous execution facility. Otherwise, if `this` stage completes normally, then the returned stage also completes normally with the same value.

Specified by:

`exceptionallyAsync` in interface `CompletionStage<T>`

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage if `this` CompletionStage completed exceptionally

Returns:

the new CompletionStage

Since:

12

exceptionallyAsync

```
public CompletableFuture<T> exceptionallyAsync
(Function<Throwable, ? extends T> fn,
 Executor executor)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` stage completes exceptionally, is executed with `this` stage's exception as the argument to the supplied function, using the supplied Executor. Otherwise, if `this` stage completes normally, then the returned stage also completes normally with the same value.

Specified by:

`exceptionallyAsync` in interface `CompletionStage<T>`

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage if `this` CompletionStage completed exceptionally

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

Since:

12

exceptionallyCompose

```
public CompletableFuture<T> exceptionallyCompose
(Function<Throwable, ? extends CompletionStage<T>> fn)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when `this` stage completes exceptionally, is composed using the results of the supplied function applied to `this` stage's exception.

Specified by:

`exceptionallyCompose` in interface `CompletionStage<T>`

Parameters:

`fn` - the function to use to compute the returned CompletionStage if `this` CompletionStage completed exceptionally

Returns:

the new CompletionStage

Since:

12

exceptionallyComposeAsync

```
public CompletableFuture<T> exceptionallyComposeAsync
(Function<Throwable, ? extends CompletionStage<T>> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when `this` stage completes exceptionally, is composed using the results of the supplied function applied to `this` stage's exception, using `this` stage's default asynchronous execution facility.

Specified by:

`exceptionallyComposeAsync` in interface `CompletionStage<T>`

Parameters:

`fn` - the function to use to compute the returned CompletionStage if `this` CompletionStage completed exceptionally

Returns:

the new CompletionStage

Since:

12

exceptionallyComposeAsync

```
public CompletableFuture<T> exceptionallyComposeAsync
(Function<Throwable, ? extends CompletionStage<T>> fn,
 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when `this` stage completes exceptionally, is composed using the results of the supplied function applied to `this` stage's exception, using the supplied Executor.

Specified by:

`exceptionallyComposeAsync` in interface `CompletionStage<T>`

Parameters:

`fn` - the function to use to compute the returned CompletionStage if `this` CompletionStage completed exceptionally

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

Since:

12

allOf

```
public static CompletableFuture<Void> allOf
(CompletableFuture<?>... cfs)
```

Returns a new CompletableFuture that is completed when all of the given CompletableFutures complete. If any of the given CompletableFutures complete exceptionally, then the returned CompletableFuture also does so, with a CompletionException holding this exception as its cause. Otherwise, the results, if any, of the given CompletableFutures are not reflected in the returned CompletableFuture, but may be obtained by inspecting them individually. If no CompletableFutures are provided, returns a CompletableFuture completed with the value `null`.

Among the applications of this method is to await completion of a set of independent CompletableFutures before continuing a program, as in: `CompletableFuture.allOf(c1, c2, c3).join();`.

Parameters:

`cfs` - the CompletableFutures

Returns:

a new CompletableFuture that is completed when all of the given CompletableFutures complete

Throws:

`NullPointerException` - if the array or any of its elements are `null`

anyOf

```
public static CompletableFuture<Object> anyOf
(CompletableFuture<?>... cfs)
```

Returns a new CompletableFuture that is completed when any of the given CompletableFutures complete, with the same result. Otherwise, if it completed exceptionally, the returned CompletableFuture also does so, with a CompletionException holding this exception as its cause. If no CompletableFutures are provided, returns an incomplete CompletableFuture.

Parameters:

`cfs` - the CompletableFutures

Returns:

a new CompletableFuture that is completed with the result or exception of any of the given CompletableFutures when one completes

Throws:

`NullPointerException` - if the array or any of its elements are `null`

cancel

```
public boolean cancel(boolean mayInterruptIfRunning)
```

If not already completed, completes this CompletableFuture with a `CancellationException`. Dependent CompletableFutures that have not already completed will also complete exceptionally, with a `CompletionException` caused by this `CancellationException`.

Specified by:

`cancel` in interface `Future<T>`

Parameters:

`mayInterruptIfRunning` - this value has no effect in this implementation because interrupts are not used to control processing.

Returns:

`true` if this task is now cancelled

isCancelled

```
public boolean isCancelled()
```

Returns `true` if this CompletableFuture was cancelled before it completed normally.

Specified by:

`isCancelled` in interface `Future<T>`

Returns:

`true` if this CompletableFuture was cancelled before it completed normally

isCompletedExceptionally

```
public boolean isCompletedExceptionally()
```

Returns `true` if this CompletableFuture completed exceptionally, in any way. Possible causes include cancellation, explicit invocation of `completeExceptionally`, and abrupt termination of a CompletionStage action.

Returns:

`true` if this CompletableFuture completed exceptionally

state

```
public Future.State state()
```

Description copied from interface: `Future`

Returns the computation state.

Specified by:

`state` in interface `Future<T>`

Returns:

the computation state

Since:

19

obtrudeValue

```
public void obtrudeValue(T value)
```

Forcibly sets or resets the value subsequently returned by method `get()` and related methods, whether or not already completed. This method is designed for use only in error recovery actions, and even in such situations may result in ongoing dependent completions using established versus overwritten outcomes.

Parameters:

`value` - the completion value

obtrudeException

```
public void obtrudeException(Throwable ex)
```

Forcibly causes subsequent invocations of method `get()` and related methods to throw the given exception, whether or not already completed. This method is designed for use only in error recovery actions, and even in such situations may result in ongoing dependent completions using established versus overwritten outcomes.

Parameters:

ex - the exception

Throws:

`NullPointerException` - if the exception is null

getNumberOfDependents

```
public int getNumberOfDependents()
```

Returns the estimated number of CompletableFutures whose completions are awaiting completion of this CompletableFuture. This method is designed for use in monitoring system state, not for synchronization control.

Returns:

the number of dependent CompletableFutures

toString

```
public String toString()
```

Returns a string identifying this CompletableFuture, as well as its completion state. The state, in brackets, contains the String `"Completed normally"` or the String `"Completed exceptionally"`, or the String `"Not completed"` followed by the number of CompletableFutures dependent upon its completion, if any.

Overrides:

`toString` in class `Object`

Returns:

a string identifying this CompletableFuture, as well as its state

newIncompleteFuture

```
public <U> CompletableFuture<U> newIncompleteFuture()
```

Returns a new incomplete CompletableFuture of the type to be returned by a CompletionStage method. Subclasses should normally override this method to return an instance of the same class as this CompletableFuture. The default implementation returns an instance of class CompletableFuture.

Type Parameters:

`U` - the type of the value

Returns:

a new CompletableFuture

Since:

9

defaultExecutor

```
public Executor defaultExecutor()
```

Returns the default Executor used for async methods that do not specify an Executor. This class uses the `ForkJoinPool.commonPool()` if it supports more than one parallel thread, or else an Executor using one thread per async task. This method may be overridden in subclasses to return an Executor that provides at least one independent thread.

Returns:

the executor

Since:

9

copy

```
public CompletableFuture<T> copy()
```

Returns a new CompletableFuture that is completed normally with the same value as this CompletableFuture when it completes normally. If this CompletableFuture completes exceptionally, then the returned CompletableFuture completes exceptionally with a CompletionException with this exception as cause. The behavior is equivalent to `thenApply(x -> x)`. This method may be useful as a form of "defensive copying", to prevent clients from completing, while still being able to arrange dependent actions.

Returns:

the new CompletableFuture

Since:

9

minimalCompletionStage

```
public CompletionStage<T> minimalCompletionStage()
```

Returns a new CompletionStage that is completed normally with the same value as this CompletableFuture when it completes normally, and cannot be independently completed or otherwise used in ways not defined by the methods of interface `CompletionStage`. If this CompletableFuture completes exceptionally, then the returned CompletionStage completes exceptionally with a CompletionException with this exception as cause.

Unless overridden by a subclass, a new non-minimal CompletableFuture with all methods available can be obtained from a minimal CompletionStage via `toCompletableFuture()`. For example, completion of a minimal stage can be awaited by

```
minimalStage.toCompletableFuture().join();
```

Returns:

the new CompletionStage

Since:

9

completeAsync

```
public CompletableFuture<T> completeAsync
(Supplier<? extends T> supplier,
 Executor executor)
```

Completes this CompletableFuture with the result of the given Supplier function invoked from an asynchronous task using the given executor.

Parameters:

`supplier` - a function returning the value to be used to complete this CompletableFuture

`executor` - the executor to use for asynchronous execution

Returns:

this CompletableFuture

Since:

9

completeAsync

```
public CompletableFuture<T> completeAsync
(Supplier<? extends T> supplier)
```

Completes this CompletableFuture with the result of the given Supplier function invoked from an asynchronous task using the default executor.

Parameters:

`supplier` - a function returning the value to be used to complete this CompletableFuture

Returns:

this CompletableFuture

Since:

9

orTimeout

```
public CompletableFuture<T> orTimeout(long timeout,
                                      TimeUnit unit)
```

Exceptionally completes this CompletableFuture with a `TimeoutException` if not otherwise completed before the given timeout elapsed.

Parameters:

`timeout` - how long to wait before completing exceptionally with a TimeoutException, in units of `unit`

`unit` - a `TimeUnit` determining how to interpret the `timeout` parameter

Returns:

this CompletableFuture

Since:

9

completeOnTimeout

```
public CompletableFuture<T> completeOnTimeout(T value,
                                              long timeout,
                                              TimeUnit unit)
```

Completes this CompletableFuture with the given value if not otherwise completed before the given timeout elapsed.

Parameters:

`value` - the value to use upon timeout

`timeout` - how long to wait before completing normally with the given value, in units of `unit`

`unit` - a `TimeUnit` determining how to interpret the `timeout` parameter

Returns:

this CompletableFuture

Since:

9

delayedExecutor

```
public static Executor delayedExecutor(long delay,
                                       TimeUnit unit,
                                       Executor executor)
```

Returns a new Executor that submits a task to the given base executor after the given delay (or no delay if non-positive). Each delay commences upon invocation of the returned executor's `execute` method.

Parameters:

`delay` - how long to delay, in units of `unit`

`unit` - a `TimeUnit` determining how to interpret the `delay` parameter

`executor` - the base executor

Returns:

the new delayed executor

Since:

9

delayedExecutor

```
public static Executor delayedExecutor(long delay,
                                       TimeUnit unit)
```

Returns a new Executor that submits a task to the default executor after the given delay (or no delay if non-positive). Each delay commences upon invocation of the returned executor's `execute` method.

Parameters:

`delay` - how long to delay, in units of `unit`

`unit` - a `TimeUnit` determining how to interpret the `delay` parameter

Returns:

the new delayed executor

Since:

9

completedStage

```
public static <U> CompletionStage<U> completedStage(U value)
```

Returns a new CompletionStage that is already completed with the given value and supports only those methods in interface `CompletionStage`.

Type Parameters:

`U` - the type of the value

Parameters:

`value` - the value

Returns:

the completed CompletionStage

Since:

9

failedFuture

```
public static <U> CompletableFuture<U> failedFuture(Throwable ex)
```

Returns a new CompletableFuture that is already completed exceptionally with the given exception.

Type Parameters:

`U` - the type of the value

Parameters:

`ex` - the exception

Returns:

the exceptionally completed CompletableFuture

Since:

9

failedStage

```
public static <U> CompletionStage<U> failedStage(Throwable ex)
```

Returns a new CompletionStage that is already completed exceptionally with the given exception and supports only those methods in interface CompletionStage.

Type Parameters:

U - the type of the value

Parameters:

ex - the exception

Returns:

the exceptionally completed CompletionStage

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.